Software Requirements Standard - (SRS)

Software Requirements Standard - (SRS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Requirements Governance

Standard ID: SLGA-SRS-001

Origin ID: OOF.PS.SLGA.01.00.01

Version: 1.0

Status: Canonical Draft · Open Standard

Effective Date: Pending Canonical Publication

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Requirements Standard (SRS) governs the identification,

formation, legitimacy, specification, completeness assessment, applicability, acceptance, consistency, change, and traceability of requirements that establish what a software system is intended, permitted, or required to accomplish.

SRS establishes the methodological conditions under which proposed software

requirements may be evaluated, recognized, and maintained as a governed basis for software creation, modification, verification, release, deployment, and lifecycle reassessment.

It distinguishes requested functionality from governed requirements,

requirement consistency from completeness, requirement acceptance from applicability, and documented requirements from evidence of operational feasibility.

SRS applies regardless of whether requirements originate from humans,

organizations, AI agents, autonomous systems, automated discovery mechanisms, or combinations of these sources.

Its principal output is:

Governed Software Requirements Baseline

Its foundational distinction is:

Requested Functionality ≠ Governed Software Requirement.

Operational Role

The operational role of SRS is:

To establish and maintain an identifiable, justified, sufficiently

complete, context-aware, accepted, and traceable requirements baseline against which software lifecycle activities can be evaluated without allowing requirement creation, modification, interpretation, or technical implementation to become an independent source of authority.

SRS is the first Parent Standard of SLGA™.

It establishes the governed basis for determining:

• What software is intended to accomplish

• Which requirements apply to the governed software object

• Where those requirements originate

• Whether requirements are sufficiently specified

• Whether material requirements gaps remain

• Whether requirements are consistent and operationally plausible

• Which requirements have been evaluated, accepted, and incorporated

• How requirements change and remain traceable throughout the software

into the applicable baseline. lifecycle.

SRS does not independently establish that software has been correctly implemented, securely configured, successfully tested, or legitimately deployed.

It establishes the requirements foundation against which subsequent

lifecycle governance activities can be assessed.

Core Question

How can software requirements remain legitimate, sufficiently complete, identifiable, consistent, applicable, accepted, and traceable as software, actors, operational contexts, and development methods change?

Position in Architecture Governance Flow

The SLGA™ Parent Standard governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance: Software Lifecycle Control

SRS governs:

Software Requirements

It is the first Parent Standard in the SLGA™ governance structure.

SRS provides the initial governed requirements baseline to subsequent

lifecycle governance domains.

The architecture flow is not a mandatory waterfall process. Requirements may be created, evaluated, accepted, and revised during any relevant lifecycle stage.

Position in the Software Lifecycle

SRS begins when a proposed software purpose, need, obligation, constraint, or requested behavior requires recognition as a software requirement.

Principal Input: Software Purpose and Requirements Inputs

Principal Output: Governed Software Requirements Baseline

SRS remains applicable throughout the software lifecycle because

requirements may be modified, superseded, suspended, or rendered inapplicable by changes in software purpose, operating conditions, dependencies, or external constraints.

A requirements baseline is therefore a governed reference state, not a permanently fixed specification.

Approved Requirements Baseline ≠ Permanently Valid Requirements Baseline.

SRS Internal Governance Flow

The canonical SRS internal governance flow is:

Software Purpose Definition → Requirements Specification → Software Scope

Management → Requirements Change Control → Requirements Traceability

These five functions correspond to the SRS Core Modules:

SPDM — Software Purpose Definition Module

2. RSPM — Requirements Specification Module 3. SSMM — Software Scope Management Module 4. RCCM — Requirements Change Control Module 5. RTM — Requirements Traceability Module

The flow establishes five distinct governance functions:

Purpose Definition: Identifies why the software exists and what

legitimate outcome it is intended to support.

Requirements Specification: Establishes sufficiently defined, evaluable,

and completeness-assessed requirements.

Scope Management: Determines the software objects, capabilities,

boundaries, contexts, and applicability conditions governed by the requirements.

Change Control: Governs the evaluation, acceptance, and incorporation of

requirements changes.

Traceability: Preserves relationships between requirements, their

origins, acceptance states, applicable conditions, subsequent lifecycle artifacts, and relevant evidence.

These functions may operate iteratively or concurrently.

Software Requirements Governance Object

Every SRS assessment must identify the software object or intended software capability to which the requirements apply.

The governed object may be:

• A software product or system

• A software component

• A software service

• An application or platform

• An autonomous software capability

• A continuously generated or self-modifying software system

• A sufficiently bounded software function or operating arrangement

The object must be identifiable independently of a particular development tool, programming language, AI model, repository, or infrastructure provider.

A requirement applicable to one software object shall not automatically be

treated as applicable to another.

Shared Functionality ≠ Shared Requirements Applicability.

Software Purpose Governance

Software requirements shall remain connected to an identifiable software

purpose.

Purpose governance establishes:

• The intended outcome or function

• The problem or operational need being addressed

• The relevant users, operators, beneficiaries, or affected

• The boundaries of the intended capability

• Material restrictions on intended behavior

• Conditions under which the stated purpose remains applicable

environments where applicable.

Software purpose provides a basis for evaluating whether proposed

requirements are relevant, coherent, and sufficiently comprehensive.

A stated purpose does not independently establish permission to implement or deploy the software.

Declared Purpose ≠ Legitimate Authority.

Requirement Origin and Legitimacy

Every material software requirement should have an identifiable origin or recognized basis.

Potential origins include:

• Authorized stakeholder instructions

• Organizational requirements

• Contractual or externally imposed obligations

• Applicable technical constraints

• Operational needs

• Security and quality requirements

• Validated observations

• Requirements proposed through automated or autonomous analysis

SRS distinguishes the source of a proposed requirement from the authority

or governance conditions necessary to recognize it as applicable.

An AI agent may discover a missing requirement, propose a requirement, identify a contradiction, or recommend a modification.

Those capabilities do not automatically authorize the agent to establish or

change the governed requirements baseline.

Requirement Proposal ≠ Requirement Authorization.

Requirement Origin ≠ Requirement Legitimacy.

General authority and delegation remain governed by competent authority

governance structures.

Requirement Identification

Material requirements shall be sufficiently identifiable to support

lifecycle traceability.

A governed requirement should preserve, where applicable:

• Requirement identity

• Requirement statement

• Origin or recognized basis

• Governed software object

• Applicable scope

• Relevant context

• Requirement classification

• Current governance state

• Acceptance status

• Version or revision

• Dependencies

• Relevant evidence references

Requirement identification shall not depend solely on its position in a

document.

Requirements may be maintained in documents, databases, structured repositories, executable specifications, model-based representations, or future equivalent forms.

Requirement Identity ≠ Document Location.

Requirement Specification

A software requirement shall be expressed with sufficient precision to

support its intended governance function.

The necessary level of detail depends on the requirement's materiality,

operational context, and consequences of incorrect interpretation.

A requirement should be sufficiently clear to determine:

• What condition or behavior is required

• To which software object it applies

• Under which circumstances it applies

• What boundaries or restrictions exist

• What would constitute evidence of satisfaction

• Whether relevant interpretation remains uncertain

SRS does not require all requirements to use a single specification language or documentation format.

However, ambiguous requirements shall not be silently interpreted as fully

specified requirements.

Documented Requirement ≠ Sufficiently Specified Requirement.

Requirement Classification

Requirements may represent different forms of governed expectations.

Functional Requirements: Required software capabilities and behavior.

Quality Requirements: Required quality characteristics or performance

conditions.

Security Requirements: Required security properties and restrictions.

Operational Requirements: Conditions relevant to intended software

operation.

Interface Requirements: Conditions governing interaction with other

systems or components.

Constraint Requirements: Restrictions imposed on software design,

behavior, execution, or operation.

Continuity Requirements: Conditions relevant to maintaining required

software capabilities over time.

Context-Specific Requirements: Requirements applicable only within defined environments, jurisdictions, deployments, or operating conditions.

Classification supports interpretation, completeness assessment, and

traceability.

It does not replace specialized governance performed by subsequent SLGA™ Parent Standards.

Requirement Applicability

A requirement shall not be presumed universally applicable merely because

it exists within a software requirements baseline.

Applicability may depend on:

• Software object

• Version or configuration

• Intended function

• Operating environment

• Deployment region

• Jurisdiction

• User or operating context

• Applicable time period

• External obligations

• Dependencies or other defined conditions

A requirement may be valid for one configuration and inapplicable to another.

Requirement Existence ≠ Requirement Applicability.

Global Requirements Record ≠ Universal Requirements Applicability.

Applicability conditions should remain identifiable and reviewable.

Requirement Priority and Obligation

SRS shall distinguish requirements according to their applicable obligation

and governance significance.

Mandatory: Required under the applicable governed conditions.

Conditional: Required when specified conditions apply.

Optional: Permitted or desirable but not mandatory under the current

governed scope.

Unresolved: Insufficiently determined for final applicability or

obligation classification.

Priority shall not be confused with obligation.

A low-priority mandatory requirement does not become optional merely because another requirement is considered more important.

Priority ≠ Obligation.

Requirements Completeness Governance

SRS shall assess whether the governed requirements baseline sufficiently

represents the declared software purpose, applicable scope, known constraints, and materially relevant operating conditions.

Requirements completeness assessment shall seek to identify:

• Missing functional requirements

• Missing quality or security requirements

• Missing operational constraints

• Unspecified failure conditions

• Missing interface or dependency requirements

• Unaddressed user or operating scenarios

• Missing requirements arising from known external obligations

• Material gaps between software purpose and documented requirements

Completeness assessment shall be proportionate to the software's governed

purpose, materiality, and operating context.

SRS does not guarantee absolute requirements completeness.

It requires material gaps and limitations of the assessment to remain

identifiable.

Requirements Consistency ≠ Requirements Completeness.

Documented Requirements ≠ Complete Requirements.

A requirements baseline may be internally consistent while omitting a condition essential to its intended purpose.

Requirements Completeness Assessment

A completeness assessment should identify the basis against which completeness is evaluated.

Relevant assessment references may include:

• Declared software purpose

• Defined software scope

• Intended operating scenarios

• Applicable constraints

• Known failure scenarios

• Material dependencies

• Identified stakeholders and affected operational contexts

• Applicable externally imposed requirements

The assessment should distinguish:

Sufficiently Covered: Available evidence supports adequate coverage for

the defined scope.

Conditionally Covered: Coverage depends on specified assumptions or

further evidence.

Material Gap Identified: A relevant requirement or condition is missing.

Unresolved: Available information is insufficient to determine coverage.

A completeness assessment shall not be represented as universal proof that no missing requirements exist.

Requirement Consistency

SRS shall identify material contradictions and incompatibilities among

requirements.

Conflicts may arise when:

• Two requirements prescribe incompatible behavior

• A functional requirement conflicts with a security restriction

• A regional requirement conflicts with a global requirement

• A new requirement contradicts an existing baseline

• Requirements impose incompatible resource or operating conditions

• Requirements depend on assumptions that cannot simultaneously hold

Conflicting requirements shall not be silently combined into a supposedly

consistent baseline.

The conflict should remain identifiable until an applicable resolution or limitation has been established.

Requirements Aggregation ≠ Requirements Consistency.

Requirements Consistency ≠ Requirements Completeness.

Requirement Conflict Resolution

Requirement conflict resolution shall preserve:

• The conflicting requirements

• Their respective origins and applicability

• The nature of the conflict

• Relevant authority and constraint boundaries

• The selected resolution or unresolved state

• The justification and supporting evidence

• The consequences for the governed baseline

SRS does not independently establish legal or institutional priority between conflicting external obligations.

Where conflict resolution exceeds the applicable authority or evidence available, the requirements state shall remain unresolved or be referred to the competent governance process.

Conflict Identification ≠ Authority to Resolve Conflict.

Software Scope Governance

Software scope determines which capabilities, functions, interfaces,

components, and operating conditions fall within the governed requirements baseline.

SRS shall distinguish:

Included Scope: Explicitly governed software capabilities or conditions.

Excluded Scope: Capabilities or conditions intentionally outside the

current requirements baseline.

Conditional Scope: Capabilities or conditions included only when specified conditions apply.

Unresolved Scope: Areas where applicability remains insufficiently established.

Scope changes may materially alter the requirements baseline and its completeness assessment.

A requirement shall not automatically expand to new software functions

merely because they are technically related.

Technical Capability ≠ Governed Scope.

Requirements Acceptance and Baseline Recognition

SRS shall distinguish requirement evaluation, requirement acceptance, and

recognition within an applicable governed requirements baseline.

A requirement may be sufficiently specified and evaluated without being accepted.

An accepted requirement may remain conditionally applicable or may not yet be incorporated into a particular baseline.

The canonical recognition sequence is:

Proposed Requirement → Evaluated Requirement → Accepted Requirement →

Applicable Baseline Requirement

Acceptance shall depend on the applicable governance conditions and

competent authority or recognition process.

Where acceptance is conditional, the relevant conditions shall remain explicit.

A requirement shall not be treated as part of an authorized baseline merely

because it was technically generated, reviewed, or successfully tested.

Evaluated Requirement ≠ Accepted Requirement ≠ Applicable Baseline

Requirement.

Requirements Acceptance Evidence

Material requirements acceptance should preserve sufficient evidence to identify:

• The requirement accepted

• The applicable acceptance conditions

• The competent acceptance authority or process

• The scope of acceptance

• The effective context or period where relevant

• Any conditions or limitations

• The baseline into which the requirement is incorporated

• The supporting acceptance record

SRS does not create the authority required for acceptance.

It governs the requirements-specific recognition state and its traceability.

Recorded Acceptance ≠ Legitimate Acceptance by default.

Requirements Baseline Formation

A Governed Software Requirements Baseline is a defined requirements state containing the applicable requirements and sufficient information to identify their scope, origin, acceptance, conditions, and governance status.

Baseline formation shall distinguish:

• Proposed requirements

• Evaluated requirements

• Accepted requirements

• Applicable baseline requirements

• Authorized baseline changes

• Superseded requirements

• Unresolved requirements

A baseline may preserve unresolved requirements or conditions, provided they are explicitly identified and not represented as satisfied or authorized for uses that require their resolution.

Baseline recognition does not establish that software already satisfies its requirements.

Governed Requirements Baseline ≠ Verified Software Conformance.

Requirements Baseline Integrity

The integrity of a requirements baseline depends on the preservation of its

meaning, scope, identity, acceptance state, applicable conditions, and material history.

SRS shall identify changes that may alter:

• Requirement meaning

• Obligation

• Applicability

• Acceptance

• Priority

• Dependencies

• Scope

• Verification criteria

• Baseline consistency

• Completeness assessment

Material changes shall not be hidden by retaining the same baseline label.

Same Baseline Label ≠ Same Requirements State.

A requirements baseline should remain reconstructable from preserved versions, changes, and evidence references.

Requirements–Operational Reality Alignment

SRS shall identify material assumptions and operating conditions upon which

software requirements depend.

Requirements may be internally consistent yet incompatible with the actual environment in which the software is intended to operate.

Relevant considerations may include:

• Infrastructure limitations

• External system dependencies

• Operating latency

• Available computational or physical resources

• Environmental conditions

• Operational capacity

• Data availability and reliability

• Interface constraints

• Known human or organizational operating limitations

SRS shall identify where requirements require operational feasibility evidence or subsequent validation.

SRS does not independently certify operational feasibility.

Relevant operational reality validation remains within competent governance processes, including ORA™ where applicable.

Requirements Consistency ≠ Operational Feasibility.

Specified Behavior ≠ Operationally Achievable Behavior.

Operational Assumption Governance

Material assumptions supporting software requirements should remain identifiable.

An operational assumption may concern:

• Expected external system behavior

• Available infrastructure

• Human intervention capability

• Operating workload

• Environmental stability

• Data quality

• Dependency availability

• Resource limits

SRS shall distinguish assumptions supported by relevant evidence from

assumptions that remain unverified.

Where an assumption is material to a requirement's applicability or feasibility, its uncertainty shall remain attached to the relevant requirements state.

A material assumption failure may trigger requirements reassessment.

Operational Assumption ≠ Verified Operational Condition.

Requirements Change Governance

Requirements may change because of:

• New software purpose

• Changed stakeholder needs

• New operating conditions

• Discovered defects or limitations

• Security or quality findings

• Changes in dependencies

• New external obligations

• Autonomous system discoveries

• Software evolution

• Retirement or migration requirements

A requirements change shall be evaluated according to its applicable scope,

authority, consequences, and effects on the existing baseline.

Material changes may require renewed completeness assessment, acceptance, and operational assumption evaluation.

SRS governs the requirements-specific change.

SCMS — Software Change Management Standard governs the broader software

lifecycle change process.

Requirements Change ≠ Automatically Authorized Software Change.

Requirements Change Impact

A material requirements change may affect:

• Existing software artifacts

• Components and dependencies

• Configurations

• Quality conditions

• Security conditions

• Release eligibility

• Deployment eligibility

• Operational continuity

• Retirement obligations

SRS shall identify relevant downstream implications without independently

performing the governance functions assigned to those downstream standards.

A requirements change may require lifecycle re-entry under SLGA™.

Requirements Approval ≠ Automatic Downstream Lifecycle Approval.

Requirements Traceability

SRS shall preserve traceable relationships between requirements and

relevant lifecycle objects.

The canonical traceability relationship is:

Software Purpose → Requirement Origin → Requirement Identity → Evaluation

→ Acceptance → Applicability → Requirements Baseline → Lifecycle Artifact / Evidence → Requirement Change History

Traceability should support questions such as:

• Why does this requirement exist?

• Who or what proposed it?

• On what basis was it accepted?

• Where does it apply?

• Which software artifacts are associated with it?

• What evidence addresses it?

• Has its meaning or applicability changed?

• Which lifecycle states may be affected by its modification?

• Which material assumptions or completeness conditions remain

unresolved?

Traceability does not itself establish that a requirement has been fulfilled.

Traceability ≠ Verification.

Autonomous Requirements Formation

SRS applies to software requirements proposed, generated, interpreted, or

modified by autonomous systems.

Autonomous requirements formation may include:

• Requirements inferred from user objectives

• Requirements derived from operational observations

• Requirements generated from existing specifications

• Requirements proposed by development agents

• Requirements created through automated system analysis

• Requirements modified during autonomous software evolution

SRS shall distinguish:

Requested Requirement → Interpreted Requirement → Proposed Requirement →

Evaluated Requirement → Accepted Requirement → Governed Requirement

An autonomous agent's interpretation of a requested outcome shall not

automatically become the authoritative requirement.

The creation of subordinate agents shall not expand requirements governance

authority.

Requirement Generation Capability ≠ Requirements Governance Authority.

Requirements Under Distributed Development

Software development may involve multiple teams, organizations, AI agents, or autonomous systems operating across different geographic regions and technical environments.

SRS shall preserve the ability to identify:

• Shared requirements

• Locally applicable requirements

• Regional requirements differences

• Conflicting requirements interpretations

• Parallel requirements changes

• Unreconciled requirements branches

• Applicable acceptance and authority conditions

• Evidence supporting baseline modifications

A locally accepted requirements change shall not automatically become part

of every other requirements baseline.

Local Requirements Acceptance ≠ Global Requirements Baseline Consistency.

Distributed requirements reconciliation shall preserve the identity and

history of material conflicting changes.

Requirements Evidence Sufficiency

SRS shall evaluate whether sufficient evidence exists to support

recognition of a requirement or requirements baseline state.

Evidence sufficiency depends on the specific governance determination.

For example:

• Evidence that a stakeholder requested a function may support

• Evidence of competent acceptance may support baseline recognition

• Evidence of a technical constraint may support requirement

• Operational evidence may support an assessment of feasibility

• Testing evidence may support downstream conformance assessment

requirement origin. applicability. assumptions.

These forms of evidence are not interchangeable.

Evidence of Request ≠ Evidence of Authorization.

Evidence of Authorization ≠ Evidence of Implementation.

Evidence of Implementation ≠ Evidence of Conformance.

Where evidence is insufficient, the relevant requirements governance condition shall remain unresolved or otherwise explicitly qualified.

Requirements Verification Conditions

SRS shall identify the conditions under which requirements can be evaluated by subsequent software quality, security, or validation processes.

This may include:

• Observable expected behavior

• Acceptance conditions

• Applicable operating context

• Relevant tolerances

• Prohibited behavior

• Required evidence

• Conditions requiring independent verification

SRS establishes the requirements-side verification basis.

It does not independently certify successful implementation or replace the testing and assessment functions of SQS or SSS.

Verifiable Requirement ≠ Verified Software.

Requirements Uncertainty

Requirements uncertainty may arise from:

• Ambiguous purpose

• Incomplete stakeholder information

• Conflicting instructions

• Missing requirements

• Uncertain operating conditions

• Unknown dependencies

• Unverified operational assumptions

• Incomplete evidence

• Unresolved external obligations

• Unpredictable autonomous system behavior

SRS shall preserve material uncertainty rather than convert assumptions

into confirmed requirements.

Uncertainty may justify further clarification, conditional recognition, restricted scope, or deferred baseline decisions.

Assumed Requirement ≠ Confirmed Requirement.

Requirements Reassessment

Requirements shall be capable of reassessment when their underlying conditions materially change.

Reassessment triggers may include:

• Changes in software purpose

• Material changes in intended use

• New or changed operating environments

• New security or quality findings

• Material dependency changes

• Changes in applicable obligations

• Requirements conflicts

• Discovery of missing material requirements

• Failure of material operational assumptions

• Autonomous changes affecting intended behavior

• Evidence that the existing baseline no longer represents the governed

software object.

Reassessment shall identify whether the current baseline remains

applicable, requires modification, or contains unresolved conditions.

Historical Requirements Approval ≠ Permanent Requirements Validity.

Requirements Continuity

Requirements governance shall remain reconstructable across changes in:

• Software creators

• Development teams

• Organizations

• AI models

• Agent systems

• Repositories

• Infrastructure providers

• Documentation formats

• Software representations

Requirements continuity depends on preservation of sufficient identity,

meaning, applicability, acceptance, change history, and supporting evidence.

Loss of the original creator or specification tool shall not automatically

invalidate every existing requirement.

However, loss of material requirements evidence shall not be treated as

proof of complete continuity.

Requirements Preservation ≠ Requirements Continuity by default.

Requirements Governance States

SRS may recognize the following requirements governance states:

Proposed: A requirement has been submitted or generated but has not

completed applicable evaluation.

Under Evaluation: The requirement is being assessed for clarity, origin, applicability, completeness relevance, consistency, or other relevant conditions.

Accepted: The requirement has satisfied the applicable acceptance

conditions.

Baseline Applicable: The accepted requirement forms part of an applicable

governed requirements baseline.

Conditional: Acceptance or applicability depends on explicitly identified

conditions.

Unresolved: Material information, authority, evidence, completeness,

feasibility, or conflict remains insufficiently determined.

Superseded: The requirement has been replaced by a subsequent governed

requirement state.

Withdrawn: The requirement has been removed from active consideration or

applicability through an appropriate governance process.

These states describe requirements governance.

They shall not be confused with software implementation, release, or

deployment states.

Minimum Governance Requirements

A conforming SRS implementation must:

• Identify the governed software object or capability

• Establish an identifiable software purpose

• Identify the origin or recognized basis of material requirements

• Distinguish requirement proposal from authorization

• Assign identifiable references to material requirements

• Define requirements with sufficient precision for their intended

• Identify applicable scope and contextual conditions

• Distinguish mandatory, conditional, optional, and unresolved

• Assess material requirements completeness against declared purpose,

• Identify material requirements gaps and completeness limitations

• Identify material requirements dependencies and conflicts

• Preserve unresolved conflicts without falsely declaring consistency

• Distinguish requirement evaluation, acceptance, and baseline

• Preserve acceptance conditions and relevant authority boundaries

• Establish an identifiable Governed Software Requirements Baseline

• Preserve requirements baseline identity and material version history

• Identify material operational assumptions and requirements requiring

• Govern requirements changes and their authorization boundaries

• Identify material downstream lifecycle implications

• Preserve traceability between purpose, requirements, acceptance,

• Identify requirements-side verification conditions

• Preserve material uncertainty and evidence limitations

• Support requirements reassessment when applicable conditions

• Preserve requirements continuity across relevant actor, technology,

• Produce a Governed Software Requirements Baseline with explicit

governance purpose. requirements. scope, and known constraints. applicability. feasibility validation. baseline, artifacts, and evidence. materially change. and organizational changes. applicable governance conditions.

Governance Logic

The canonical SRS governance logic is:

Identify Governed Software Object

↓

Establish Software Purpose

↓

Identify Requirement Origins and Applicable Authority Boundaries

↓

Identify Proposed Requirements

↓

Define Requirement Identity and Specification

↓

Determine Scope and Contextual Applicability

↓

Classify Obligation and Priority

↓

Assess Requirements Completeness

↓

Identify Material Requirements Gaps

↓

Evaluate Requirements Consistency and Dependencies

↓

Identify and Address Material Conflicts

↓

Identify Material Operational Assumptions

↓

Identify Requirements Requiring Operational Feasibility Validation

↓

Assess Evidence Sufficiency and Uncertainty

↓

Evaluate Requirements Acceptance Conditions

↓

Establish Requirements Governance States

↓

Form Governed Software Requirements Baseline

↓

Preserve Requirements Traceability

↓

Evaluate Requirements Changes and Downstream Impacts

↓

Reassess Completeness, Acceptance, and Applicability When Material Changes Occur

↓

Maintain Requirements Baseline Integrity and Continuity

The principal output is:

Governed Software Requirements Baseline

Governance Outputs

SRS may produce:

- Software Purpose Record

- Requirements Governance Object - Requirement Origin Record - Requirement Identity Record - Requirements Specification - Requirements Classification Record - Requirements Applicability State - Requirements Scope Record - Requirements Obligation State - Requirements Priority Record - Requirements Completeness Assessment - Material Requirements Gap Record - Requirements Consistency Assessment - Requirements Conflict Record - Requirements Conflict Resolution State - Requirements Evaluation State - Requirements Acceptance Record - Requirements Acceptance Conditions - Governed Software Requirements Baseline - Requirements Baseline Version - Operational Assumption Record - Requirements Operational Feasibility Validation Need - Requirements Change Record - Requirements Change Impact Record - Requirements Traceability Record - Requirements Verification Conditions - Requirements Evidence Sufficiency State - Requirements Uncertainty Record - Requirements Reassessment Record - Requirements Continuity Record - Requirements Governance State

The principal output is:

Governed Software Requirements Baseline

Critical Distinctions

SRS preserves the following distinctions:

- Requested Functionality ≠ Governed Software Requirement

- Declared Purpose ≠ Legitimate Authority - Requirement Proposal ≠ Requirement Authorization - Requirement Origin ≠ Requirement Legitimacy - Requirement Identity ≠ Document Location - Documented Requirement ≠ Sufficiently Specified Requirement - Documented Requirements ≠ Complete Requirements - Requirements Consistency ≠ Requirements Completeness - Requirement Existence ≠ Requirement Applicability - Priority ≠ Obligation - Requirements Aggregation ≠ Requirements Consistency - Conflict Identification ≠ Authority to Resolve Conflict - Technical Capability ≠ Governed Scope - Evaluated Requirement ≠ Accepted Requirement ≠ Applicable Baseline Requirement - Recorded Acceptance ≠ Legitimate Acceptance by default - Governed Requirements Baseline ≠ Verified Software Conformance - Same Baseline Label ≠ Same Requirements State - Requirements Consistency ≠ Operational Feasibility - Specified Behavior ≠ Operationally Achievable Behavior - Operational Assumption ≠ Verified Operational Condition - Requirements Change ≠ Automatically Authorized Software Change - Requirements Approval ≠ Automatic Downstream Lifecycle Approval - Traceability ≠ Verification - Requirement Generation Capability ≠ Requirements Governance Authority - Local Requirements Acceptance ≠ Global Requirements Baseline Consistency - Evidence of Request ≠ Evidence of Authorization - Evidence of Authorization ≠ Evidence of Implementation - Evidence of Implementation ≠ Evidence of Conformance - Verifiable Requirement ≠ Verified Software - Assumed Requirement ≠ Confirmed Requirement - Historical Requirements Approval ≠ Permanent Requirements Validity

Cross-Architecture Boundary

SRS governs software-specific requirements within SLGA™.

It does not independently govern:

• General authority and delegation

• Organizational accountability

• Autonomous system behavior

• General validation methodology

• Operational reality validation

• Software implementation

• Software quality assessment

• Software security assessment

• Release authorization

• Deployment authorization

• General legal or regulatory interpretation

SRS interoperates with:

GOA™ — Governance Architecture: General authority and delegation.

AGA™ — Accountability Governance Architecture: Responsibility and

accountability.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous system

behavior and execution boundaries.

ORA™ — Operational Reality Architecture: Operational reality and

feasibility conditions.

VALIDOS™: Independent validation governance.

ADIT®: Evidence and auditability governance.

Within SLGA™, SRS provides requirements governance states to subsequent Parent Standards.

It does not replace the governance functions of SCS, SDS, SCFS, SQS, SSS, SRLS, SDPS, SCMS, SRTS, or SLCS.

Canonical Boundary

SRS begins when a proposed software purpose, requirement, obligation, constraint, or requested behavior requires recognition within the software lifecycle.

SRS governs:

Software purpose, requirement origin, requirement identification,

specification, classification, applicability, obligation, priority, completeness assessment, material requirements gaps, consistency, conflict identification, conflict resolution conditions, software scope, requirements evaluation, requirements acceptance, baseline recognition, requirements baseline formation, baseline integrity, operational assumptions, requirements-side operational feasibility validation needs, requirements changes, requirements change impact, requirements traceability, requirements-side verification conditions, uncertainty, reassessment, continuity, and requirements governance states.

SRS does not grant authority to create, approve, release, deploy, or operate software.

It does not establish that requirements have been successfully implemented.

It does not certify operational feasibility, software quality, or security.

It does not authorize software lifecycle transitions governed by other SLGA™ Parent Standards.

SRS ends at the establishment and maintenance of an identifiable,

sufficiently assessed, applicable, accepted, traceable, and governable:

Governed Software Requirements Baseline

This baseline becomes an input to subsequent software lifecycle governance.

Canonical Closing Statement

Software development does not begin with code.

It begins with a purpose and the requirements through which that purpose becomes an identifiable obligation, capability, or constraint.

A requirement may be written by a person, generated by an AI agent, derived from operational evidence, or imposed through an external obligation.

Its origin alone does not establish its legitimacy.

Its presence in a document does not establish its applicability.

Its internal consistency does not establish its completeness.

Its acceptance does not establish its implementation.

And its technical specification does not establish that the required behavior is achievable under actual operating conditions.

Requested Functionality ≠ Governed Software Requirement.

Requirements Consistency ≠ Requirements Completeness.

Evaluated Requirement ≠ Accepted Requirement ≠ Applicable Baseline

Requirement.

Requirements Consistency ≠ Operational Feasibility.

Requirements Approval ≠ Verified Software Conformance.

As software becomes increasingly autonomous, distributed, and continuously

modified, requirements governance must remain independent of the technologies and actors through which software is developed.

SRS does not promise perfect requirements.

It establishes the conditions under which requirements can be identified, justified, assessed for completeness, accepted, changed, challenged, and traced without losing their meaning or governance boundaries.

SRS establishes the governed requirements foundation upon which software

lifecycle decisions can be justified — ensuring that software development begins not merely with what has been requested, but with requirements whose purpose, legitimacy, completeness, applicability, acceptance, and limitations can be meaningfully governed.

Module Architecture

→ SPDM — Software Purpose Definition Module

→ RSPM — Requirements Specification Module

→ SSMM — Software Scope Management Module

→ RCCM — Requirements Change Control Module

→ RTM — Requirements Traceability Module

Related Documents

→ About Software Requirements Standard (SRS)